

Progress Report 4

Student: Michael Hein (300375535)

Course: CSIS 4495 – Applied Research Project

Project: Real-Time Cryptocurrency Streaming Pipeline

Reporting Period: Mar 9 – Mar 17, 2026

1. Work Date/Hours Logs

Date	Number of Hours	Description of Work Done
Mar 10, 2026	1.5	Implemented price threshold alerts — configurable <code>ALERT_PRICE_THRESHOLDS</code> (e.g., <code>BTC-USD:above:100000</code>) to trigger Slack/email when a symbol crosses above or below a price. Updated <code>src/utils/alerts.py</code> and <code>config.py</code> .
Mar 10, 2026	1	Removed WebSocket dependency from dashboard (switched to polling-only), fixed favicon path, and updated <code>config/env.example</code> with new alert variables. Code checked into repo.
Mar 10, 2026	0.5	Added unit tests for price threshold config parsing in <code>tests/test_config.py</code> .
Mar 12, 2026	2	Implemented CI/CD — GitHub Actions workflows: <code>test.yml</code> (pytest, ruff lint/format on Python 3.11/3.12) and <code>docker-build.yml</code> (build all Docker images). Added <code>.dockerignore</code> and <code>ruff.toml</code> .
Mar 12, 2026	2	Created Dockerfiles for producer, consumer, dashboard, S3-sink, and Redshift-sink. Extended <code>docker-compose.yml</code> to run full pipeline (Kafka, producer, consumer, dashboard) with optional AWS profile.
Mar 12, 2026	1.5	Added new dashboard visualizations: exchange price comparison (grouped bar chart), exchange filter (dropdown + pills), KPI cards, configurable refresh rate and time window. Updated <code>api_server.py</code> , <code>index.html</code> , <code>app.js</code> , <code>styles.css</code> .
Mar 12, 2026	1.5	Implemented ML anomaly detection (<code>src/utils/anomaly.py</code>) — Isolation Forest on <code>trade_count</code> , <code>volatility</code> , <code>volume</code> , <code>price_change</code> . Configurable via <code>ALERT_ANOMALY_ENABLED</code> and <code>ANOMALY_CONTAMINATION</code> . Integrated into dashboard API.
Mar 12, 2026	1	Updated exchange adapters and sinks for Docker compatibility (env vars, health checks). Fixed tests for new config and mocked services.

Date	Number of Hours	Description of Work Done
Mar 15, 2026	2	Implemented Prometheus metrics (<code>src/utils/metrics.py</code>) — counters for trades published, Kafka errors, dashboard requests; gauges for events, freshness, consumer status. Instrumented producer, consumer, and dashboard.
Mar 15, 2026	1.5	Added Prometheus scrape config (<code>config/prometheus.yml</code>) and Grafana provisioning — datasource, pre-built Crypto Pipeline dashboard (trades/min, data freshness, API latency, pipeline health).
Mar 15, 2026	1	Integrated Prometheus and Grafana into <code>docker-compose.yml</code> . Updated README with observability section and metrics table.

Total Hours This Period: ~15.5

2. Summary Description of Work Done

This reporting period focused on four main areas: (1) enhanced alerting with price thresholds, (2) CI/CD and containerization, (3) ML-based anomaly detection and dashboard improvements, and (4) observability with Prometheus and Grafana.

Price Threshold Alerts: I extended the alerting system to support configurable price thresholds (e.g., `BTC-USD:above:100000`). When a symbol crosses above or below a configured price, the pipeline triggers Slack and/or email alerts. I also removed the WebSocket dependency from the dashboard in favor of polling-only to simplify the architecture.

CI/CD and Docker: I set up GitHub Actions for automated testing (pytest on Python 3.11 and 3.12, ruff lint and format checks) and Docker image builds. The pipeline can now run end-to-end via `docker compose up`, with separate Dockerfiles for the producer, consumer, dashboard, and AWS sinks. This improves reproducibility and deployment.

Anomaly Detection and Dashboard: I implemented an Isolation Forest-based anomaly detector that flags unusual patterns in trade count, volatility, volume, and price change per symbol. The detector uses a rolling window and is configurable via environment variables. The dashboard gained new visualizations: exchange price comparison, exchange filter pills, KPI cards, and configurable refresh rate and time window.

Observability: I added Prometheus metrics across the pipeline (trades published, Kafka errors, dashboard requests, data freshness, consumer status) and integrated Prometheus and Grafana into Docker Compose. A pre-provisioned Grafana dashboard displays trades per minute, data freshness, API latency, and pipeline health.

One issue encountered was Grafana's requirement for a specific datasource format — this was resolved by using provisioning YAML to auto-configure the Prometheus datasource and dashboard on startup.

3. Repo Check-In of Implementation Completed

The following files/folders have been checked into the repo since the last progress report:

New files:

- `src/utls/metrics.py` — Prometheus metrics definitions (counters, gauges, histograms)
- `src/utls/anomaly.py` — Isolation Forest-based anomaly detection
- `config/prometheus.yml` — Prometheus scrape configuration
- `config/grafana/provisioning/datasources/datasource.yml` — Grafana Prometheus datasource
- `config/grafana/provisioning/dashboards/dashboard.yml` — Dashboard provisioning
- `config/grafana/dashboards/crypto-pipeline.json` — Pre-built Crypto Pipeline dashboard
- `.github/workflows/test.yml` — pytest + ruff CI
- `.github/workflows/docker-build.yml` — Docker image build CI
- `Dockerfile.producer`, `Dockerfile.consumer`, `Dockerfile.dashboard`, `Dockerfile.s3-sink`, `Dockerfile.redshift-sink`
- `.dockerignore`, `ruff.toml`

Modified files:

- `Implementation/dashboard/api_server.py` — Anomaly detection, price alerts, Prometheus instrumentation, new endpoints
- `Implementation/dashboard/web/index.html`, `app.js`, `styles.css` — Exchange comparison, filter pills, KPI cards, configurable UI
- `Implementation/producer/base_exchange.py`, `multi_exchange_producer.py` — Prometheus metrics
- `Implementation/consumer/spark_stream.py` — Prometheus metrics
- `Implementation/athena/s3_sink.py`, `Implementation/redshift/redshift_sink.py` — Docker compatibility
- `Implementation/producer/exchange_*.py`, `Implementation/quicksight/setup_quicksight.py` — Env/config updates
- `docker-compose.yml` — Full pipeline + Prometheus + Grafana
- `config/env.example` — Price thresholds, anomaly config
- `src/utls/alerts.py`, `config.py` — Price threshold logic
- `README.md` — Observability section, metrics table, Docker instructions
- `requirements.txt` — `prometheus_client`
- `tests/test_config.py`, `test_exchanges.py`, `test_quicksight.py`, `test_sinks.py` — Updated for new config and mocks

4. AI Use Section

AI Tool Name	Version, Account Type	Specific Feature for Which the AI Tool Was Used	Value Addition
--------------	-----------------------	---	----------------

AI Tool Name	Version, Account Type	Specific Feature for Which the AI Tool Was Used	Value Addition
Cursor (Claude)	Claude-3.5-Sonnet / Premium	Price threshold alert design — config format and parsing logic	Validated regex and edge cases; integrated with existing Slack/email alert flow
Cursor (Claude)	Claude-3.5-Sonnet / Premium	GitHub Actions workflows — test.yml and docker-build.yml structure	Adapted for multi-Python-version matrix; verified ruff and pytest commands locally
Cursor (Claude)	Claude-3.5-Sonnet / Premium	Isolation Forest anomaly detection — feature selection and sklearn usage	Chose features (trade_count, volatility, volume, price_change); tuned contamination parameter; tested on sample data
Cursor (Claude)	Claude-3.5-Sonnet / Premium	Prometheus metrics — counter/gauge/histogram definitions and labels	Mapped metrics to actual code paths; ensured labels matched scrape config
Cursor (Claude)	Claude-3.5-Sonnet / Premium	Grafana dashboard JSON — panel layout and PromQL queries	Customized queries for pipeline-specific metrics; verified datasource provisioning
Cursor (Claude)	Claude-3.5-Sonnet / Premium	Docker Compose multi-service setup	Tested full stack locally; resolved port and dependency ordering

5. Appendix: AI Prompt History

Prompt 1 — Price threshold alerts

"I need to add configurable price threshold alerts — when BTC-USD goes above 100000 or ETH-USD goes below 3000, send Slack/email. What format should I use for the config and how do I integrate with the existing alerts module?"

Prompt 2 — CI/CD setup

"Set up GitHub Actions for my Python project: run pytest on push/PR, and run ruff check and format. Support Python 3.11 and 3.12."

Prompt 3 — Docker Compose full pipeline

"I want to run the entire crypto pipeline with docker compose — Kafka, producer, consumer, dashboard. Create Dockerfiles for each component and update docker-compose.yml."

Prompt 4 — Anomaly detection

"Add ML-based anomaly detection to the dashboard — detect unusual trade patterns per symbol. Use sklearn Isolation Forest. What features should I use from the metrics?"

Prompt 5 — Prometheus metrics

"Add Prometheus metrics to the pipeline: trades published per exchange, Kafka errors, dashboard request count and latency, data freshness. Where should I expose the /metrics endpoint?"

Prompt 6 — Grafana dashboard

"Create a Grafana dashboard for the crypto pipeline metrics. I want to see trades per minute, data freshness, API latency, and whether the consumer is running. Use provisioning so it auto-loads."

Prompt 7 — Grafana datasource error

"Grafana says 'datasource not found' when loading the dashboard. My prometheus.yml scrape config targets localhost:9093. How do I fix the provisioning?"